NAME :G.AKSHAYA

REG NO: 192521156

COURSE CODE : CSA0619

COURSE NAME : DESIGN AND ANALYSIS OF ALGORITHMS

CO5 - ASSESSMENT TOOL – 01

Q2. Exam Timetabling (Graph Coloring / NP-Complete Problem)

Aim:

To develop a conflict-free university examination timetable using Graph Coloring and Backtracking, ensuring that students do not have overlapping examinations and that available time slots and rooms are used efficiently.

Problem Statement:

A university needs to schedule examinations for different subjects. No student should have two examinations in the same time slot. The timetable must also consider time-slot availability, room capacity, and student conflicts. Graph coloring with backtracking can be used to assign suitable time slots to each examination.

Objective:

- Create a conflict-free examination timetable.
- Avoid overlapping examinations for the same student.
- Assign available time slots to examinations.
- Consider limited room availability.
- Use graph coloring to represent examination conflicts.
- Apply backtracking to find a feasible timetable.
- Verify that all constraints are satisfied.

Graph Representations :

Each exam is represented as a vertex.

An edge is created between two exams if they have common students.

Each color represents a time slot.

In the graph:

- **Vertex** → Represents an examination.
- **Edge** → Represents a conflict between two examinations.
- **Color** → Represents an examination time slot.

Example:

E1

/ \

E2----E3

E4

If E1, E2 and E3 have student conflicts, they must receive different colors/time slots.

For example:

Color 1 → Morning

Color 2 → Afternoon

Color 3 → Next Day Morning

Working of Graph Coloring :

The algorithm assigns a color to each examination.

First, an examination is selected and assigned an available time slot. The next examination is checked against already scheduled examinations. If there is no conflict, the time slot is assigned. If a conflict occurs, another time slot is tried.

If no available time slot satisfies the constraints, the algorithm **backtracks** to the previous examination and changes its assigned slot.

Constraints:

1. Student Conflict

Two examinations containing common students cannot be scheduled in the same time slot.

2. Time-Slot Constraint

Every examination must be assigned to one of the available time slots.

3. Room Capacity

The assigned room must have enough capacity for the students attending the examination.

4. Room Availability

A room cannot be used by two examinations during the same time slot.

5. Unique Assignment

Each examination must have exactly one valid time-slot assignment.

Backtracking Approach :

1. Select an examination.
2. Try an available time slot.
3. Check all conflicts.
4. If the slot is valid, assign it.
5. Move to the next examination.
6. If no valid slot exists, go back to the previous examination.
7. Change its time slot and continue.
8. Repeat until all examinations are successfully scheduled.

Example:

Suppose there are four examinations:

E1, E2, E3, E4

If the conflicts are:

E1 ↔ E2

E1 ↔ E3

E2 ↔ E3

E2 ↔ E4

E3 ↔ E4

Algorithm:

1. Start.
2. Read the number of examinations and available time slots.
3. Create the conflict graph.
4. Select an examination.
5. Assign an available time slot.
6. Check whether the assignment satisfies all constraints.
7. If valid, continue with the next examination.
8. If invalid, try another time slot.
9. If no slot is possible, perform backtracking.
10. Continue until all examinations are scheduled.
11. Display the final timetable.
12. Stop.

SOURCE CODE:

main.py Visualize Run

```
def is_safe(exam, slot, graph, timetable):
    for other in range(len(graph)):
        if graph[exam][other] == 1 and timetable[other] == slot:
            return False
    return True

def graph_coloring(exam, slots, graph, timetable):
    if exam == len(graph):
        return True

    for slot in range(slots):
        if is_safe(exam, slot, graph, timetable):
            timetable[exam] = slot

            if graph_coloring(exam + 1, slots, graph, timetable):
                return True

            timetable[exam] = -1

    return False

```

```
22
23 n = int(input("Enter number of exams: "))
24 slots = int(input("Enter number of time slots: "))
25
26 print("Enter conflict matrix:")
27 graph = []
28
29 for i in range(n):
30     graph.append(list(map(int, input().split())))
31
32 timetable = [-1] * n
33
34 if graph_coloring(0, slots, graph, timetable):
35     print("\nExam Timetable:")
36     for i in range(n):
37         print("Exam", i + 1, "-> Time Slot", timetable[i] + 1)
38 else:
39     print("No feasible timetable exists.")
```

OUTPUT:

```
Output

Enter number of exams: 4
Enter number of time slots: 3
Enter conflict matrix:
0 1 1 0
1 0 1 1
1 1 0 1
0 1 1 0

Exam Timetable:
Exam 1 -> Time Slot 1
Exam 2 -> Time Slot 2
Exam 3 -> Time Slot 3
Exam 4 -> Time Slot 1

=== Code Execution Successful ===
```

Verification:

- Exam 1 and Exam 4 can use Time Slot 1 because there is no conflict between them.
- Exam 2 uses Time Slot 2.
- Exam 3 uses Time Slot 3.
- No two conflicting exams are assigned to the same time slot.
- Therefore, the timetable is conflict-free.

Advantages :

- Avoids examination conflicts.
- Provides a systematic timetable.
- Backtracking can find a feasible assignment when one exists.
- Graph representation makes conflicts easy to identify.
- Can handle different numbers of time slots.

Limitations :

- Graph Coloring is an NP-Complete problem.
- Backtracking can require considerable time for large numbers of exams.
- Performance decreases as the number of exams and conflicts increases.
- Room capacity and multiple rooms require additional constraints.

Complexity Analysis:

For n exams and m time slots, the worst-case backtracking complexity is approximately:

Time: $O(m^n)$

Space: $O(n)$

Result:

The examination timetable was successfully generated using Graph Coloring and Backtracking. Each examination was assigned a time slot while ensuring that conflicting examinations were not scheduled simultaneously. The approach is suitable for small and medium-sized timetabling problems, while larger universities may require more advanced optimization techniques.